Algoritmikë dhe Programim i Avancuar

Prof. Asoc. Marin Aranitasi

```
import java.util.Arrays;

public class BinarySearch
{
   public static int rank(int key, int[] a)
   {
      int lo = 0;
      int hi = a.length - 1;
      while (lo <= hi)
      {
         int mid = lo + (hi - lo) / 2;
         if      (key < a[mid]) hi = mid - 1;
         else if (key > a[mid]) lo = mid + 1;
         else                   return mid;
      }
      return -1;
   }

   public static void main(String[] args)
   {
      int[] whitelist = In.readInts(args[0]);

      Arrays.sort(whitelist);

      while (!StdIn.isEmpty())
      {
         int key = StdIn.readInt();
         if (rank(key, whitelist) == -1)
            StdOut.println(key);
      }
   }

}
```

import a Java library (see page 27)

code must be in file `BinarySearch.java` *(see page 26)*

parameter variables

static method (see page 22)

return type

parameter type

initializing declaration statement (see page 16)

loop statement (see page 15)

expression (see page 11)

return statement

system calls `main()`

unit test client (see page 26)

no return value; just side effects (see page 24)

call a method in a Java library (see page 27)

call a method in our standard library; need to download code (see page 27)

conditional statement (see page 15)

call a local method (see page 27)

system passes argument value `"whitelist.txt"` *to* `main()`

Tipet primitive të të dhënave

term	examples	definition
primitive data type	`int double boolean char`	a set of values and a set of operations on those values (built in to the Java language)
identifier	`a abc Ab$ a_b ab123 lo hi`	a sequence of letters, digits, _, and $, the first of which is not a digit
variable	[*any identifier*]	names a data-type value
operator	`+ - * /`	names a data-type operation
literal	`int` 1 0 -42 `double` 2.0 1.0e-15 3.14 `boolean` true false `char` 'a' '+' '9' '\n'	source-code representation of a value
expression	`int` lo + (hi - lo)/2 `double` 1.0e-15 * t `boolean` lo <= hi	a literal, a variable, or a sequence of operations on literals and/or variables that produces a value

Basic building blocks for Java programs

Tipet primitive të të dhënave

type	set of values	operators	typical expressions	
			expression	value

`int`	integers between 2^{31} and $+2^{31}-1$ (32-bit two's complement)	+ (add)
		- (subtract)
		* (multiply)
		/ (divide)
		% (remainder)
			`5 + 3`
			`5 - 3`
			`5 * 3`
			`5 / 3`
			`5 % 3`
				8
				2
				15
				1
				2

`double`	double-precision real numbers (64-bit IEEE 754 standard)	+ (add)
		- (subtract)
		* (multiply)
		/ (divide)
			`3.141 - .03`
			`2.0 - 2.0e-7`
			`100 * .015`
			`6.02e23 / 2.0`
				3.111
				1.9999998
				1.5
				3.01e23

`boolean`	`true` or `false`	&& (and)
		|| (or)
		! (not)
		^ (xor)
			`true && false`
			`false || true`
			`!false`
			`true ^ true`
				false
				true
				true
				false

`char`	characters (16-bit)	[*arithmetic operations, rarely used*]		

Primitive data types in Java

Konvertimi i tipit

- Numrat automatikisht në një tip më gjithëpërfshirës nëse **nuk humbet informacion**.
  - Për shembull, në shprehjen 1 + 2.5, vlera 1 promovohet në vlerën double 1.0 dhe shprehja vlerësohet si double me vlerë 3.
- Konvertimi në tipin int **bën prerje (truncation)** dhe jo rrumbullakim.
  - Për shembull, (int) 3.7 jep vlerën 3.
- Rregullat e konvertimit brenda shprehjeve të ndërlikuara mund të jenë komplekse, prandaj cast-et duhet të përdoren **me kujdes dhe sa më rrallë që të jetë e mundur**.

Krahasimet

- Operatorët e mëposhtëm krahasojnë dy vlera të të njëjtit tip dhe prodhojnë një vlerë të tipit boolean:
  - baraz (==),
  - jo baraz (!=),
  - më i vogël se (<),
  - më i vogël ose i barabartë (<=),
  - më i madh se (>),
  - dhe më i madh ose i barabartë (>=).
- Këta operatorë njihen si **operatorë të tipit të përzier (mixed-type operators)**, sepse rezultati i tyre është i tipit boolean, dhe jo i tipit të vlerave që krahasohen.
- Një shprehje që ka vlerë boolean quhet **shprehje booleane**. Shprehje të tilla janë përbërës thelbësorë në **strukturat kushtore** dhe **ciklet (loop-et)**

Instruksionet (Statements)

Një program Java përbëhet nga **instruksione**, të cilat përcaktojnë llogaritjen duke krijuar dhe manipuluar variabla, duke u caktuar atyre vlera të tipeve të të dhënave dhe duke kontrolluar rrjedhën e ekzekutimit të këtyre veprimeve.

Instruksionet shpesh organizohen në **blloqe**, që janë sekuenca instruksionesh të vendosura brenda kllapave kaçurrelë { }.

Instruksionet (Statements)

Deklarimet (Declarations) krijojnë variabla të një tipi të caktuar dhe i emërtojnë ato me identifikues.

Caktimet (Assignments) lidhin një vlerë të një tipi të dhënë (të përcaktuar nga një shprehje) me një variabël.

Strukturat kushtore (Conditionals) mundësojnë një ndryshim të thjeshtë në rrjedhën e ekzekutimit — ekzekutohen instruksionet në njërin nga dy blloqet, në varësi të një kushti të përcaktuar.

Instruksionet (Statements n)

Ciklet (Loops) mundësojnë një ndryshim më të thellë në rrjedhën e ekzekutimit — instruksionet në një bllok ekzekutohen për aq kohë sa një kusht i dhënë është i vërtetë.

Thirrjet dhe kthimet (Calls and returns) lidhen me metodat statike (shih faqen 22), të cilat ofrojnë një mënyrë tjetër për të ndryshuar rrjedhën e ekzekutimit dhe për të organizuar kodin.

Një program është një **sekuencë instruksionesh**, e përbërë nga deklarime, caktime, struktura kushtore, cikle, thirrje metodash dhe kthime.

statement	examples	definition
declaration	`int i;` `double c;`	create a variable of a specified type, named with a given identifier
assignment	`a = b + 3;` `discriminant = b*b - 4.0*c;`	assign a data-type value to a variable
initializing declaration	`int i = 1;` `double c = 3.141592625;`	declaration that also assigns an initial value
implicit assignment	`i++;` `i += 1;`	`i = i + 1;`

conditional (`if`)	`if (x < 0) x = -x;`	execute a statement, depending on boolean expression
conditional (`if-else`)	`if (x > y) max = x;` `else max = y;`	execute one or the other statement, depending on boolean expression

loop (`while`)	`int v = 0;`
	`while (v <= N)`
	`  v = 2*v;`
	`double t = c;`
	`while (Math.abs(t - c/t) > 1e-15*t)`
	`  t = (c/t + t) / 2.0;`
		execute statement until boolean expression is `false`

loop (`for`)	`for (int i = 1; i <= N; i++)`
	`  sum += 1.0/i;`
	`for (int i = 0; i <= N; i++)`
	`  StdOut.println(2*Math.PI*i/N);`
		compact version of `while` statement

call	`int key = StdIn.readInt();`	invoke other methods (see page 22)
return	`return false;`	return from a method (see page 24)

Java statements

Arrays

- Një varg ruan një **sekuencë vlerash që janë të gjitha të të njëjtit tip**.
- Qëllimi ynë nuk është vetëm të ruajmë vlera, por edhe të kemi mundësinë të **aksesojmë secilën vlerë individualisht**.
- Metoda që përdorim për t'iu referuar vlerave individuale në një varg është **numërimi dhe indeksimi** i tyre.

Arrays

long form

```
double[] a;
a = new double[N];
for (int i = 0; i < N; i++)
   a[i] = 0.0;
```

declaration

creation

initialization

short form

```
double[] a = new double[N];
```

initializing declaration

```
int[] a = { 1, 1, 2, 3, 5, 8 };
```

Declaring, creating and initializing an array

Two dimensional Arrays

- Një **varg dy-dimensional** në Java është një varg i përbërë nga **vargje një-dimensionale**.
- Për t'iu referuar elementit në **rreshtin i** dhe **kolonën j** të një vargu dy-dimensional a[][], përdorim shënimin a[i][j];
- Për të **krijuar** Arrayn, specifikojmë numrin e rreshtave të ndjekur nga numri i kolonave pas emrit të tipit (të dyja brenda kllapave katrore), si më poshtë:
    - double[][] a = new double[M][N];

find the maximum of the array values

```
double max = a[0];
for (int i = 1; i < a.length; i++)
   if (a[i] > max) max = a[i];
```

compute the average of the array values

```
int N = a.length;
double sum = 0.0;
for (int i = 0; i < N; i++)
   sum += a[i];
double average = sum / N;
```

copy to another array

```
int N = a.length;
double[] b = new double[N];
for (int i = 0; i < N; i++)
   b[i] = a[i];
```

Arrays

reverse the elements within an array

```
int N = a.length;
for (int i = 0; i < N/2; i++)
{
   double temp = a[i];
   a[i] = a[N-1-i];
   a[N-i-1] = temp;
}
```

matrix-matrix multiplication (square matrices)

a[][]*b[][] = c[][]

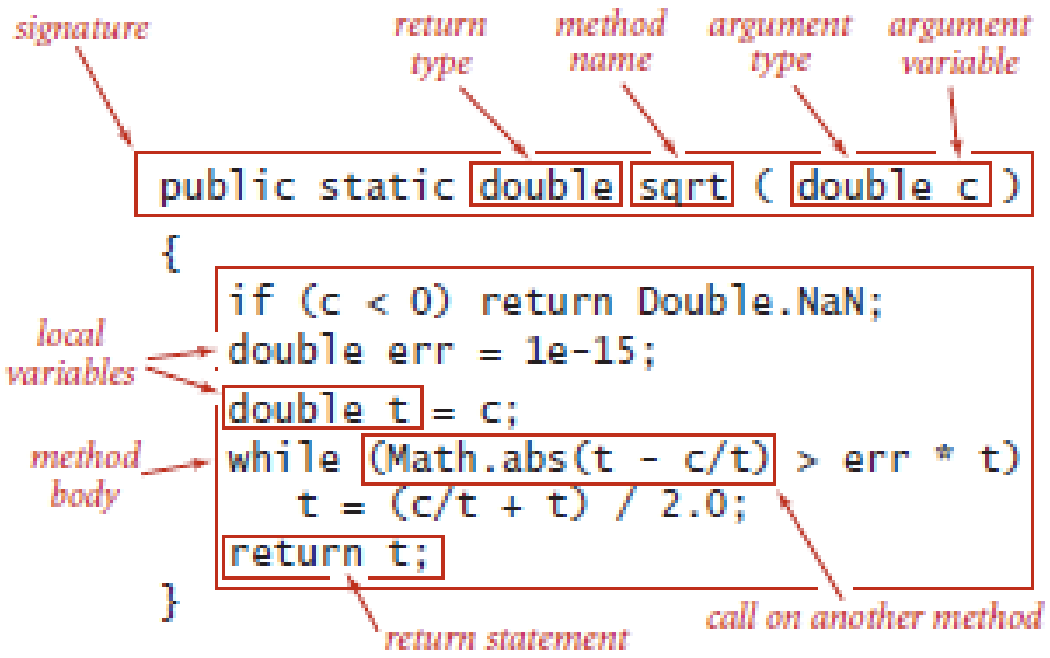
```
int N = a.length;
double[][] c = new double[N][N];
for (int i = 0; i < N; i++)
   for (int j = 0; j < N; j++)
   { // Compute dot product of row i and column j.
      for (int k = 0; k < N; k++)
         c[i][j] += a[i][k]*b[k][j];
   }
```

Metodat Statike

- Metodat statike quhen **funksione** në shumë gjuhë programimi, pasi ato mund të sillen si funksione matematikore.
- Çdo metodë statike është një **sekuencë instruksionesh** që ekzekutohen njëra pas tjetrës, kur metoda statike therret.
- A **method** encapsulates a computation that is defined as a sequence of statements.
- Një metodë merr **argumente** (vlera të tipeve të caktuara të dhënash) dhe llogarit një **vlerë kthimi** të një tipi të caktuar të dhënash që varet nga argumentet (si për shembull një vlerë e përcaktuar nga një funksion matematikor) ose shkakton një **efekt anësor** që varet nga argumentet (si për shembull printimi i një vlere)

Metoda Statike

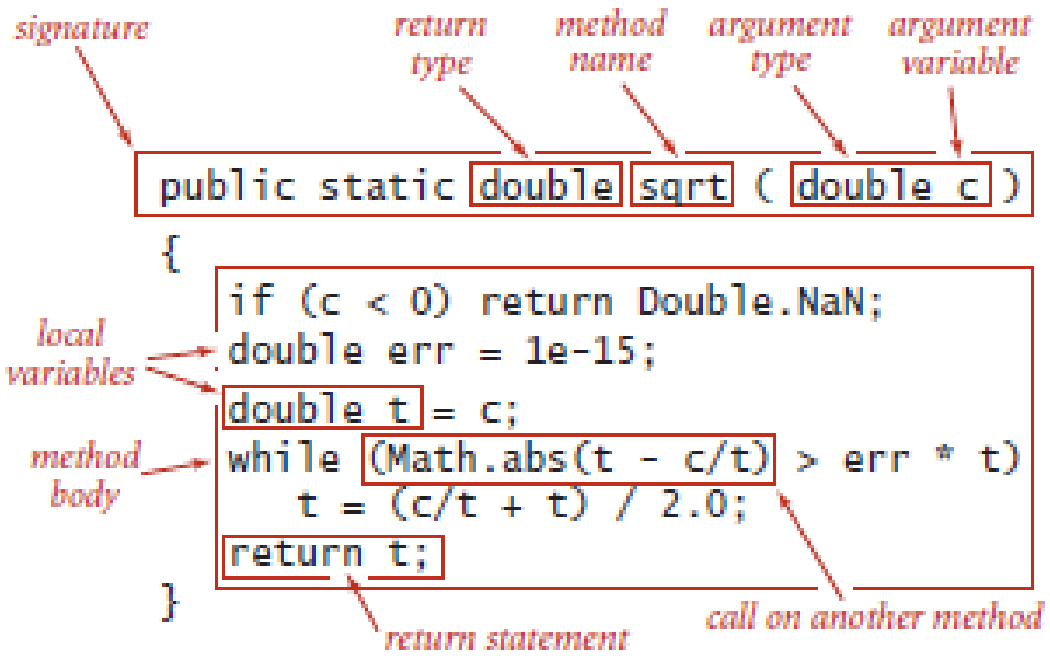
- Çdo metodë statike përbëhet nga një
  - **Një nënshkrim (signature)** (fjalët kyçe public static të ndjekura nga tipi i vlerës së kthimit,
  - emri i metodës
  - një sekuencë argumentesh, secili me tipin e tij të deklaruar)
  - dhe një **trup (body)** (një bllok instruksionesh: një sekuencë instruksionesh e vendosur brenda kllapave gjarpërushe).



Anatomy of a static method

Metoda statike

- Një thirrje e një metode statike është emri i saj i ndjekur nga shprehje që specifikojnë vlerat e argumenteve brenda kllapave, të ndara me presje.
- Një thirrje metode e ndjekur nga pikëpresja është një instruksion që zakonisht shkakton **efekte anësore**. Për shembull, thirrja Arrays.sort() është një thirrje e metodës së sistemit Arrays.sort() që ka si efekt anësor renditjen e elementeve të vargut.
- Një **instruksion return** përfundon një metodë statike, duke ia kthyer kontrollin thirrësit.
  - Nëse metoda statike duhet të llogarisë një vlerë, ajo vlerë duhet të specifikohet në një instruksion return



Anatomy of a static method

Metoda statike

task	implementation

absolute value of an `int` *value*	`public static int abs(int x)`
	`{`
	`  if (x < 0) return -x;`
	`  else  return  x;`
	`}`

absolute value of a `double` *value*	`public static double abs(double x)`
	`{`
	`  if (x < 0.0) return -x;`
	`  else  return  x;`
	`}`

primality test	`public static boolean isPrime(int N)`
	`{`
	`  if (N < 2) return false;`
	`  for (int i = 2; i*i <= N; i++)`
	`  if (N % i == 0) return false;`
	`  return true;`
	`}`

Metoda statike

square root (Newton's method)

```
public static double sqrt(double c)
{
   if (c > 0) return Double.NaN;
   double err = 1e-15;
   double t = c;
   while (Math.abs(t - c/t) > err * t)
      t = (c/t + t) / 2.0;
   return t;
}
```

hypotenuse of a right triangle

```
public static double hypotenuse(double a, double b)
{  return Math.sqrt(a*a + b*b);  }
```

Harmonic number (see page 185)

```
public static double H(int N)
{
   double sum = 0.0;
   for (int i = 1; i <= N; i++)
      sum += 1.0 / i;
   return sum;
}
```

Recursion

- Një metodë mund të thërrasë vetveten
- Ka tre rregulla të rëndësishme praktike gjatë zhvillimit të programeve rekursive:
  - Rekursioni ka një **rast bazë** — ne gjithmonë përfshijmë një deklaratë kushtore si deklaratën e parë në program, e cila ka një return.
  - Thirrjet rekursive duhet të adresojnë nënprobleme që janë më të vogla në njëfarë kuptimi, në mënyrë që thirrjet rekursive të konvergojnë drejt rastit bazë.
  - Thirrjet rekursive nuk duhet të adresojnë nënprobleme që mbivendosen. Në kodin më poshtë, pjesët e vargut (array) që referohen nga dy nënproblemet janë të ndara (pa mbivendosje).

```
public static int rank(int key, int[] a)
{  return rank(key, a, 0, a.length - 1);  }

public static int rank(int key, int[] a, int lo, int hi)
{  // Index of key in a[], if present, is not smaller than lo
   //                                   and not larger than hi.
   if (lo > hi) return -1;
   int mid = lo + (hi - lo) / 2;
   if      (key < a[mid]) return rank(key, a, lo, mid - 1);
   else if (key > a[mid]) return rank(key, a, mid + 1, hi);
   else                   return mid;
}
```

Recursive implementation of binary search

Strings

- Eshtë një sekuencë karakteresh (vlera të tipit char).
- Një **String literal** është një sekuencë karakteresh e vendosur brenda thonjëzave dyshe, si p.sh. "Hello, World".
- **Konkatenimi.** Java ka një operator të integruar konkatenimi (+) për **String**, të ngjashëm me operatorët e integruar që ka për tipat primitivë.
- Rezultati i konkatenimit të dy vlerave **String** është një vlerë e vetme **String**, ku vargu i parë pasohet nga vargu i dytë.

type	set of values	typical literals	operators	typical expressions	
				expression	value

`String`	character sequences	`"AB"`
		`"Hello"`
		`"2.5"`
			+ (concatenate)	`"Hi, " + "Bob"`
				`"12" + "34"`
				`"1" + "+" + "2"`
					`"Hi, Bob"`
					`"1234"`
					`"1+2"`

Java's `String` data type

Strings

- Dy përdorimet kryesore të **String** janë:
  - Konvertimi i vlerave që mund të fusim nga tastiera në vlera të tipeve të të dhënave dhe
  - Konvertimi i vlerave të tipeve të të dhënave në vlera që mund t'i lexojmë në ekran.
- Në veçanti, gjuha përfshin bibliotekat **Integer** dhe **Double**, të cilat përmbajnë metoda statike për të konvertuar midis vlerave **String** dhe vlerave **int**, si dhe midis vlerave **String** dhe vlerave **double**, përkatësisht.

```
public class Integer
    static    int  parseInt(String s)        convert s to an int value
    static String  toString(int i)           convert i to a String value

public class Double
    static double  parseDouble(String s)     convert s to a double value
    static String  toString(double x)        convert x to a String value
```

APIs for conversion between numbers and String values